Training *is* a Monte Carlo Estimate

From counting samples to the divergence a language model descends

One could sum up this whole document in a sentence:

> A language model's training loss is an average over examples; that average is an estimate of an expectation; and that expectation, once you subtract a constant the model cannot change, is the *distance from the data's distribution to the model's*. Training walks that distance downhill.

This note assumes nothing about entropy, cross-entropy, or KL-divergence — it builds all three from scratch. It is a companion to *Next-Token Prediction is Geometry* (where the loss $L = -\log p_y$ comes from) and *The Radon–Nikodym Theorem* (where the density ratio at the heart of the distance comes from).

What a Monte Carlo estimate is

Suppose you want the average value of some quantity that depends on a random outcome — the *expectation*. If a random thing x takes values with probabilities P, and $g(x)$ is a number computed from each outcome, the expectation is the probability-weighted average

$$\mathbb{E}_{x\sim P}[g(x)] \;=\; \sum_{x} P(x)\, g(x)$$

(a sum for a discrete x, an integral $\int g\, dP$ for a continuous one). Often this sum is impossible to do directly — too many terms, or P is known only as a thing you can *sample* from, not a formula you can add up.

The Monte Carlo trick is to stop trying to compute it and instead *draw samples and average*:

$$\mathbb{E}_{x\sim P}[g(x)] \;\approx\; \frac{1}{N}\sum_{i=1}^{N} g(x_i), \qquad x_1, \ldots, x_N \text{ drawn independently from } P.$$

The law of large numbers promises this average converges to the true expectation as $N \to \infty$. That is the whole method: *to estimate an average over a distribution, average over samples from it.*

> **Insight** The canonical picture is estimating π by throwing darts at a square with an inscribed circle: you cannot easily integrate the circle's area by hand, but the *fraction* of random darts landing inside it tends to $\pi/4$. No formula was evaluated — outcomes were sampled and counted. Keep this in mind: everything below is darts, thrown at language.

A detour: codes and wasted bits

Before averaging surprise, it helps to know what surprise *is* in concrete terms: the length of a message.

Imagine transmitting a long text one symbol at a time, spending as few bits as possible. You agree in advance on a *code* — a dictionary assigning each possible symbol a string of bits — and the art is to give *frequent* symbols *short* codewords and rare ones long codewords. Shannon's source coding theorem makes the optimal choice exact: a symbol occurring with probability q should be given a codeword of length

$$\ell(x) = -\log_2 q(x) \text{ bits.}$$

Common symbols (q near 1) cost almost nothing; rare ones (q near 0) cost a great deal; an impossible one ($q = 0$) cannot be sent at all, the length $-\log_2 0 = \infty$.

A code is therefore *built for a distribution*: fix the frequencies q you expect and the codeword lengths follow. Here is the whole idea in one example. Take a code tuned for the symbol frequencies of **English** and use it to transmit a text that is actually **Indonesian**. Every symbol still gets sent, but the lengths are now wrong for this text: symbols common in Indonesian yet rare in English have been handed long codewords, so the message runs *longer than it needed to*. Had you used the code built for Indonesian, the same text would have been shorter. That difference — the *extra bits per symbol, averaged over the Indonesian text* — is pure waste, caused entirely by encoding with the wrong distribution.

That waste is what this document is really about. The three sections that follow name its pieces precisely:

- the message length under the *right* code is the **entropy**;
- the length under the *wrong* code is the **cross-entropy**;
- the excess of the second over the first is the **KL-divergence**.

And the per-symbol ideal length $-\log_2 q(x)$ is exactly the *surprise* $-\log q$ that the loss is built from: a model that assigns the true next token a probability is, bit for bit, a *code* for language, and its loss is the length of the message it would send.

> **Insight** Real integer-length codes — genuine Huffman codes — cannot spend a *fractional* $-\log_2 q$ bits on a symbol; they round to whole bits, paying up to nearly one extra bit per symbol on top of the waste above. So "excess = KL" is exact only for *ideal* fractional-length coding — arithmetic coding, or Huffman applied to long blocks, where the rounding amortises to nothing. Huffman is the right picture; the clean equality lives in the long-text limit. One unit note: coding theory counts in *bits* ($\log_2$), while the rest of this document counts in *nats* ($\ln$) — the same quantities scaled by $\ln 2 \approx 0.693$, so 1 nat ≈ 1.44 bits.

The training loss is already a sample average

From the companion geometry note, a single training example — a context x and the token y that actually followed — costs the model

$$L = -\log q(y \mid x),$$

where $q(\cdot \mid x)$ is the model's predicted distribution over the next token (its softmax). Read $-\log q(y \mid x)$ as a *surprise* — equivalently, by the previous section, as the *code length* the model spends on the token that truly came next: it is zero when the model was certain of the right answer ($q = 1$), and grows without bound as the model's probability for the true token shrinks toward zero.

Training minimises the average surprise over the whole corpus of N examples:

$$\widehat{L}(\theta) \;=\; \frac{1}{N}\sum_{t=1}^{N} -\log q_\theta(y_t \mid x_t).$$

Now compare this to the Monte Carlo formula. The examples (x_t, y_t) are *samples drawn from the true distribution of language*, call it P_{data}. The quantity being averaged is $g(x, y) = -\log q_\theta(y \mid x)$. So $\widehat{L}$ is a Monte Carlo estimate, and what it estimates is the expectation

$$\widehat{L}(\theta) \;\longrightarrow\; \mathbb{E}_{(x,y)\sim P_{\text{data}}}\big[-\log q_\theta(y \mid x)\big].$$

The darts are the training examples; the target is the average surprise of the model across all of language. We now name and dissect that target.

Expected surprise: cross-entropy

Give that expectation a name. For any "true" distribution P and any model distribution Q, the *cross-entropy* is the average surprise the model Q suffers on outcomes drawn from P:

$$H(P, Q) \;=\; \mathbb{E}_{x\sim P}\big[-\log q(x)\big] \;=\; -\sum_x p(x)\log q(x).$$

This is the "wrong code" length of the detour made exact: the average bits spent sending text drawn from P through the code built for Q. It is precisely what the training loss estimates, with $P = P_{\text{data}}$ and $Q = Q_\theta$. So:

The training loss is a Monte Carlo estimate of the cross-entropy $H(P_{data}, Q_\theta)$.

Cross-entropy measures two things tangled together: how unpredictable the data is in the first place, and how badly the model's guesses are aligned with it. The next two sections pull those two apart.

The floor: entropy

What is the *smallest* the average surprise could possibly be? Intuitively, a model does best when it predicts with the true probabilities themselves — when $Q = P$. Setting $Q = P$ in the cross-entropy gives the *entropy* of P:

$$H(P) \;=\; H(P, P) \;=\; -\sum_x p(x)\log p(x).$$

This is the "right code" length of the detour: the average bits spent when the code is built for the very distribution producing the text — the shortest any code can achieve. It is the irreducible average surprise: the uncertainty built into the data, which no model however good can talk its way beneath. If the next word is genuinely a coin-flip between two options, no predictor achieves less than one bit of surprise on average.

That $H(P)$ really is a floor — that no Q beats the truth — is a short, classic argument. The logarithm is *concave*, which by Jensen's inequality means $\mathbb{E}[\log Z] \le \log \mathbb{E}[Z]$ for any positive random Z. Apply it to $Z = q(x)/p(x)$ with $x \sim P$:

$$\mathbb{E}_P\left[\log \frac{q(x)}{p(x)}\right] \;\le\; \log \mathbb{E}_P\left[\frac{q(x)}{p(x)}\right] \;=\; \log \sum_x p(x)\,\frac{q(x)}{p(x)} \;=\; \log \sum_x q(x) \;=\; \log 1 \;=\; 0.$$

So $\mathbb{E}_P[\log q] \le \mathbb{E}_P[\log p]$, which rearranges to $H(P, Q) \ge H(P)$, with equality exactly when $q = p$ everywhere. The truth is the unique best predictor of itself.

The gap: KL-divergence

Cross-entropy is at least the entropy; define the *gap between them* as the *Kullback–Leibler divergence*:

$$D_{\mathrm{KL}}(P \,\|\, Q) \;=\; H(P, Q) - H(P).$$

Substituting the two definitions and combining the sums collapses it to a single, memorable form:

$$D_{\mathrm{KL}}(P \,\|\, Q) \;=\; -\sum_x p(x) \log q(x) + \sum_x p(x) \log p(x) \;=\; \sum_x p(x) \log \frac{p(x)}{q(x)} \;=\; \mathbb{E}_P\left[\log \frac{p(x)}{q(x)}\right].$$

Read it directly: it is the average, over outcomes that truly occur, of the log-ratio between how likely they really are and how likely the model thinks they are. Two properties, both already proved above:

- $D_{\mathrm{KL}}(P\|Q) \geq 0$ always (it is the gap above a floor), and
- $D_{\mathrm{KL}}(P\|Q) = 0$ if and only if $Q = P$ exactly.

So it behaves like a distance from P to Q: non-negative, and zero only when they coincide. It is the detour's wasted bits made exact — the extra length from encoding P-text with the Q-code, averaged over the text. (It is not symmetric — $D_{\mathrm{KL}}(P\|Q) \neq D_{\mathrm{KL}}(Q\|P)$ in general — so it is a directed discrepancy, not a true metric. The coding picture says why: encoding Indonesian text with an English code wastes a different number of bits than English text with an Indonesian code, because the waste is averaged over whichever language is really being spoken — and that language is the first argument P. Which way round it points will matter in the last section.)

Insight The three quantities form one sentence: **cross-entropy = entropy + divergence**, i.e. total average surprise = unavoidable surprise + avoidable surprise. Entropy is the part of the difficulty that belongs to language; KL-divergence is the part that belongs to the model's mistakes. Training can only touch the second.

Putting it together: training descends the divergence

Now assemble the pieces. The loss estimates cross-entropy, and cross-entropy splits as

$$\underbrace{H(P_{\text{data}}, Q_\theta)}_{\text{what the loss estimates}} \;=\; \underbrace{H(P_{\text{data}})}_{\text{constant in } \theta} \;+\; \underbrace{D_{\mathrm{KL}}(P_{\text{data}} \,\|\, Q_\theta)}_{\text{all the } \theta\text{-dependence}}.$$

The entropy of language $H(P_{\text{data}})$ does not depend on the model's parameters θ at all — it is a fixed number, the irreducible floor. So it is a constant offset, and *its gradient is zero*. Minimising the cross-entropy and minimising the divergence are therefore the same optimisation: same minimiser, same gradient at every step.

Training a language model is Monte Carlo gradient descent on $D_{\mathrm{KL}}(P_{\text{data}} \,\|\, Q_\theta)$. The loss you actually compute — the average surprise over a batch of examples — is a sampled estimate of the cross-entropy, which equals the divergence plus the fixed entropy of language. The constant is unknowable but irrelevant: the gradient erases it. Every step pulls the model's distribution Q_θ a little closer to the data's P_{data}, measured by KL.

There is a loose end worth naming. The divergence's own definition contains $\log \frac{p_{\text{data}}}{q_\theta}$ — but p_{data}, the true probability of a sentence, is unknown. How can training descend a quantity it cannot even evaluate? Because the unknown half, $\mathbb{E}_P[\log p_{\text{data}}]$, is exactly $-H(P_{\text{data}})$, the constant. Monte Carlo evaluates only the half it can — the average of $-\log q_\theta$ over samples — and the half it cannot is the constant whose gradient vanishes. You descend the divergence by computing only the part of it that moves.

Two layers of sampling

"Monte Carlo" is in force twice over, at two scales, and they are worth keeping distinct:

- **Mini-batch → corpus.** A training step does not use all N examples; it averages over a small random batch. That batch average is a (noisier, smaller-N) Monte Carlo estimate of the full-corpus average. This is the "stochastic" in stochastic gradient descent — the estimator is noisy, and the noise is the batch-to-batch variance.
- **Corpus → language.** The corpus is itself a finite sample from the true distribution of language. So the corpus average is a Monte Carlo estimate of the *population* cross-entropy. The gap between "low loss on the corpus" and "low divergence from real language" is where overfitting and generalisation live.

The inner layer is about variance; the outer layer is about generalisation. Both are the same formula — average over samples to estimate an expectation — applied at different scales.

Where Radon–Nikodym enters

The companion note develops this in full; here is the hinge. The divergence's integrand is a *ratio of densities*, p/q. The theorem guaranteeing such a ratio exists as a genuine function — rather than meaningless notation — is Radon–Nikodym, and the ratio is its derivative, written dP/dQ. The divergence is, in its exact form,

$$D_{\text{KL}}(P \,\|\, Q) = \mathbb{E}_P\left[\log \frac{dP}{dQ}\right].$$

In ordinary pretraining this stays in the background: because the samples are drawn *from* P_{data} directly, the ratio never has to be computed — only q_θ is evaluated, and the rest is the constant. Radon–Nikodym underwrites the definition without ever being touched.

It becomes something you *compute* the moment the samples come from the wrong distribution. If you want an average over P but can only draw from some other Q, you reweight each sample by the ratio:

$$\mathbb{E}_P[g] = \mathbb{E}_Q\left[g \cdot \frac{dP}{dQ}\right].$$

That reweighting factor dP/dQ is the Radon–Nikodym derivative made into arithmetic — the technique called *importance sampling*. It is exactly what appears in reinforcement-style fine-tuning (where samples drawn under an old policy are reused to estimate averages under the current one), and the directed, asymmetric nature of KL noted earlier is why the ratio points the way it does. That thread — where the derivative stops being foundational and starts doing the work — is the subject of the Radon–Nikodym note.